FSD
Semester 1, 2026

Week 06

useContext *again*

useReducer

Asynchronous call *again*

Unit testing

RMIT UNIVERSITY

Segment 1

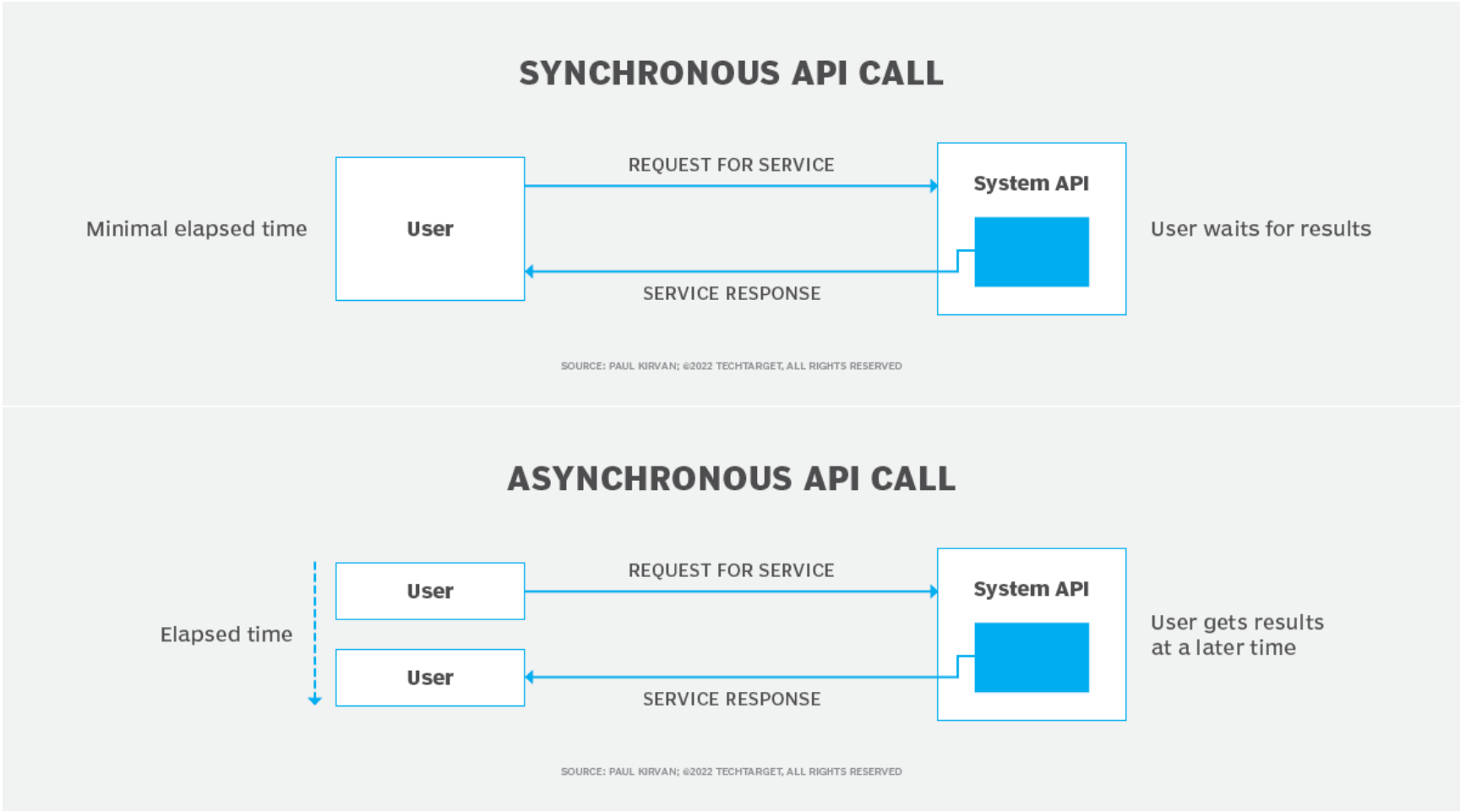
Assignment 1 Discussion / Reminder
Review API concepts
useContext *again*

A1 FAQs and discussion forum

- FAQs
- Discussion forum

RMIT

Review Sync vs Asyc API Calls



https://www.techtarget.com/whatis/definition/synchronous-asynchronous-API

RMIT

Endpoints

Google Books API provides several **REST**ful **endpoints** to search for volumes, retrieve specific book metadata, and manage personal libraries

What is an endpoint?
It allows your client application to send a request server function via a URL.

What is REST API?

- Representational State Transfer Application Programming Interface
- An architectural style for designing networked application.
- REST APIs use standard HTTP methods to perform Create Read Update Delete operations (CRUD).

RMIT

Rest APIs Core Operations

Web requests are done with these methods understood by HTTP:

Operation	HTTP method	Purpose	Examples
Create	POST	Create a new resource	Signing up a new user Create a book record
Review	GET	Retrieves data from a resource	Viewing a profile Retrieve a book metadata
Update	PUT / PATCH	Updates an existing resource	Changing an email address Update book detail
Delete	DELETE	Remove a resource	Deleting a post Delete a book record

RMIT

API using Custom Hooks

- One of the very apt uses of a custom hook could be a data fetch scenario
- Imagine an *asynchronous call to an API*- a very typical scenario in a web application
- Let us call a free API- look at these Google APIs https://developers.google.com/apis-explorer
- We will call the Book API https://developers.google.com/books/docs/v1/reference/
- You have text box which can list books from google store based on what you type on it. If no book available on that particular search query than show 'No Book Found'.

Google Book API endpoints

These endpoints allow you to search and view information about public books:

Search for Volumes: Perform a full-text search using keywords like title, author, or ISBN.

- `GET https://www.googleapis.com/books/v1/volumes?q={search_terms}`

Get Volume Details: Retrieve detailed metadata for a specific book using its unique Volume ID.

- `GET https://www.googleapis.com/books/v1/volumes/{volumeId}`

RMIT

But how are asynchronous operations handled in React?

- *useEffect* is a good hook to deal with scenarios such as: calls to an API, fetch data, etc.
  1. It's generally best practice to define a function that you call from your effect and then do your async operations there. It allows you to use React's keywords such as *async/await*
  2. There are other ways to code async operation- use a library known as *react query*
     - https://tanstack.com/query/latest/docs/framework/react/overview
     - It provides additional hooks that you can play around with

Custom hook that does an asynchronous fetch

- Time to look at an example: Lectorial05 example6

Search : react search

Book Title : React.Js Essentials

Book Title : React Projects

Book Title : React and React Native

Book Title : React and React Native

Book Title : Learning React

Book Title : Fullstack React

Book Title : Learning React

Book Title : React Cookbook

Book Title : React: Up & Running

Book Title : Pro MERN Stack

```
function useAsyncHook(searchBook) {
  const [result, setResult] = React.useState([]);
  const [loading, setLoading] = React.useState("false");

  React.useEffect(() => {
    async function fetchBookList() {
      try {
        setLoading("true");
        const response = await fetch(
          `https://www.googleapis.com/books/v1/volumes?q=${searchBook}
        );
```

useContext *revisited*

- Looking back at what we have covered re this hook
  - Use this hook when you need to share data globally like current authenticated user, theme, or preferred language, *etc*.
  - Context is primarily used when some data needs to be accessible by many components at different nesting levels.
  - For example, consider a Page component that passes a user and avatarSize prop several levels down so that deeply nested Link and Avatar components can read it-

```
<Page user={user} avatarSize={avatarSize} />
// ... which renders ...
<PageLayout user={user} avatarSize={avatarSize} />
// ... which renders ...
<NavigationBar user={user} avatarSize={avatarSize} />
// ... which renders ...
<Link href={user.permalink}>
  <Avatar user={user} size={avatarSize} />
</Link>
```

useContext *revisited*

- You're not limited to a single child for a component. You may pass to multiple children
- This pattern is sufficient for many cases when you need to decouple a child from its immediate parents.
- So, remember
  - If same data needs to be accessible by many components in the tree, and at different nesting levels.
    - Context lets you "broadcast" such data, and changes to it, to all components below.
- According to Context API we need
    - A Context object.
    - A Context provider.
    - A Context consumer.

useContext- updating context

- It is often necessary to update the context from a component that is nested somewhere deeply in the component tree.
- In this case you can pass a function down through the context to allow consumers to update the context.
- Time to look at an example where we will update context

  - example1: emp- salary example

useContext- *practical use*

- A very good use of this hook is to *change themes* on a web page
- You could create *light* and *dark themes* and switch between them
- Have a look at these interesting implementations in your *free time*:
  - [https://www.nicknish.co/blog/react-hooks-deep-dive-usecontext]
  - [https://www.codewithlinda.com/blog/dark-mode-with-react-context/]

useContext

- Some apt applications of Context hook can be
  - Theming—Pass down app theme
  - Internationalization—Pass down translation messages
  - Authentication—Pass down current authenticated user
- *Gotchas*
  - You should not be reaching for context to solve every state sharing problem
  - If your state is frequently updated, React Context may not be an efficient choice
  - Context does NOT have to be global to the whole app, but can be applied to one part of your tree
  - You can (and probably should) have multiple logically separated contexts in your app.
  - React Context is an excellent API for apps with infrequent state changes, but it can quickly turn into a developer's nightmare if not used correctly.

Segment 2

useReducer hook Asynchronous *again* Retaining values upon refresh;

useReducer *hook*

- React documentation states that - useReducer is an alternative to useState when state logic is complex.
- useReducer is usually preferable to useState when you have complex state logic that involves multiple sub-values or when the next state depends on the previous one.
- useReducer also lets you optimise performance for components that trigger deep updates because you can pass dispatch down instead of callbacks.
- In turn, hooks like useContext and useReducer eliminate the dependency on Redux for many React apps

Reducer is a JS concept!

- JavaScript *reducer* is one of the most useful array methods
- It is a cleaner way of iterating over and processing the data stored in an array.
- The reduce method accepts two arguments: a reducer function for the array that is used as a callback and an optional initialValue argument. The reducer function takes four arguments: accumulator, currentValue, currentIndex, and array.
- Look at the example at:
  - https://www.w3schools.com/jsref/tryit.asp?filename=tryjsref_reduce2

useReducer *hook*

How it is used?

```
const [state, dispatch] = useReducer(reducer,
                                     initialState);
```

- It accepts a reducer function with the application initial state, returns the current application state, then dispatches a function.
- An alternative to useState
  - Accepts a reducer of type
    - (state, action) => newState
  - and returns the current state paired with a dispatch method.

useReducer *hook*

- Time to look at an example
  - example2: *a to do list example with useReducer & useContext*

- You are advised to pay undivided attention while this example is being discussed
  - That is where you will realise the power of these hooks.

Lectorial Exercise

- Can you think of some scenarios where useReducer will be a much better choice than useState?

useReducer *versus* Redux: *future discussion*

- Something we will come back in the forthcoming lectorials
- Redux creates one global state container which is above your whole application and is called a store WHILE useReducer creates an independent component co-located state container within your component.
- useReducer is often co-used with useContext to deal with complex state management
- According to React documentation-
  - having all your state handled by React and not using *third party library like Redux* will make your code easier to understand and work with React Hooks will make building components much faster with less code.

Asynchronous *again*

- We are back to asynchronous operations
- In week 5, we looked at an example of custom hook that made an asynchronous call to Google's Book API using JavaScript keywords *async-await*
- In JavaScript, there is yet another way to make an asynchronous call
  - Using *then/catch* and *finally*
  - This is left for your self exploration
- According to JS documentation
  - then and catch and finally are methods of the Promise object, and they are chained one after the other.
  - Each takes a callback function as its argument and returns a Promise.

Asynchronous *again*

- Often, using *chained then* methods can require difficult alterations
- By contrast, async/await lent itself to a more readable solution
- Adding many then methods can easily force us further down the path towards callback issues
- When you have a choice, always use async/await as compared to then/catch

Asynchronous *again*

- There is a third-party library *react-query* that you can use in react web apps
- React query
- https://tanstack.com/query/latest/docs/framework/react/overview
- It makes it easy to handle asynchronous UI states
- React Query consists of a main useQuery hook
- Let us look at one simple example
  - example3: retrieve movie info from SWAPI - Star Wars API (https://pipedream.com/apps/swapi)

How to persist state upon page refresh?: *a common malady*

- Sometimes you may want to keep the state of a React component persistent after a browser refresh.
- A simple way to accomplish this without having to rely on any third-party library, is to use the localStorage API together with useEffect hook.
  - *Note: there are other ways too*
  - If you want to reuse a component across an application, this is not a good approach!!
- Time for an example
  - example4

Hooks Hooks Hooks

- Looking back we have covered
  1. useState
  2. useEffect
  3. Custom hooks
  4. useRef
  5. useContext
  7. useMemo
  8. useCallback
  9. useReducer: *we will revisit*

Unit testing

- *Love & hate relationship with developers*
- *You will find blogs which totally shun them* and then some who will revere them
- As a student and future graduate of CS/IT/SE discipline, do not develop strong anti/for opinions- *you cannot afford to at the start of your career.* It will sound obnoxious and may delay your career progression
- You need to learn the concept and implementation involving unit tests for different languages and framework(s)
- *Why are unit tests developed?*

Unit testing

- Unit tests exercise very small parts of the application in complete isolation, comparing their actual behaviour with the expected
- Think of unit tests as small code segments that exercise your application, interacting with tiny portions of it
- They offer various advantages
  - help you find bugs earlier
  - become a safety net
  - might contribute to high quality code & better application architecture
  - can act as part of documentation
  - detect *code smells* in your codebase

Unit testing- when it goes wrong?

- Too many
- It becomes a ritual where it is more about them and less about codebase
- They are complex
- They duplicate logic
- They are not readable
- They are deterministic (*the test should always present the same behaviour if no changes were made to the code under test*)
- Unit tests start interfering with implementation (*need to be concerned with behaviour*)

Next week

- Assignment 1 due: Sunday, 19.04.2025 (11:59 pm Melbourne time)
- Week 07 topics:
  - Database programming and ORM
  - Writing your own APIs
  - Server-side using Node & Express.js
  - Remaining hooks